Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Technology: SkyWater SKY130 (130 nm), standard cell library sky130_fd_sc_hd

Tools: Icarus Verilog, GTKWave, Yosys, OpenSTA, LibreLane (OpenROAD, Magic, KLayout, Netgen)

SUBMITTED BY

Name	Roll Number
K. Phalgun	2026702011
Adabala Naveen	2026702001

Repository: https://github.com/phalgun18/mult4_rtl_to_gdsii

Contents

Section	Title
1	Design Overview
2	Part A: RTL Design and Verification
3	Part B: Logic Synthesis with Yosys
4	Part C: Static Timing Analysis with OpenSTA
5	Part D: Physical Design, RTL to GDSII
6	Answers to the Report Questions (Q1 to Q6)

1. Design Overview

Item	Detail
Design	4-bit x 4-bit unsigned array multiplier (8-bit product)
Top module	mult_array
Technology	SkyWater SKY130, sky130_fd_sc_hd
Registers	4-bit a register, 4-bit b register, 8-bit p register (16 flip-flops)
Reset	Asynchronous, active low (rst_n)
Latency	2 clock cycles
Throughput	1 result per clock cycle
Array logic	16 AND gates and 12 full adders

Ports of mult_array

Port	Direction	Width	Description
clk	input	1	Clock, rising edge
rst_n	input	1	Asynchronous reset, active low
a	input	4	Multiplicand
b	input	4	Multiplier
p	output	8	Registered product a x b

Tool flow

Stage	Tool	Part
RTL and gate-level simulation	Icarus Verilog	A, B
Waveform viewing	GTKWave	A
Logic synthesis	Yosys	B
Static timing analysis	OpenSTA	C
RTL to GDSII	LibreLane (OpenROAD, Magic, KLayout, Netgen)	D

2. Part A: RTL Design and Verification

2.1 Files

File	Content
rtl/full_adder.v	Module full_adder (a, b, cin, sum, cout) written with assign statements
rtl/mult_array.v	Module mult_array, 12 full_adder instances, no * operator
tb/tb_mult.v	Exhaustive self-checking testbench, 256 vectors, writes mult.vcd

2.2 Full adder equations

Output	Equation
sum	a ^ b ^ cin
cout	(a & b) | (cin & (a ^ b))

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

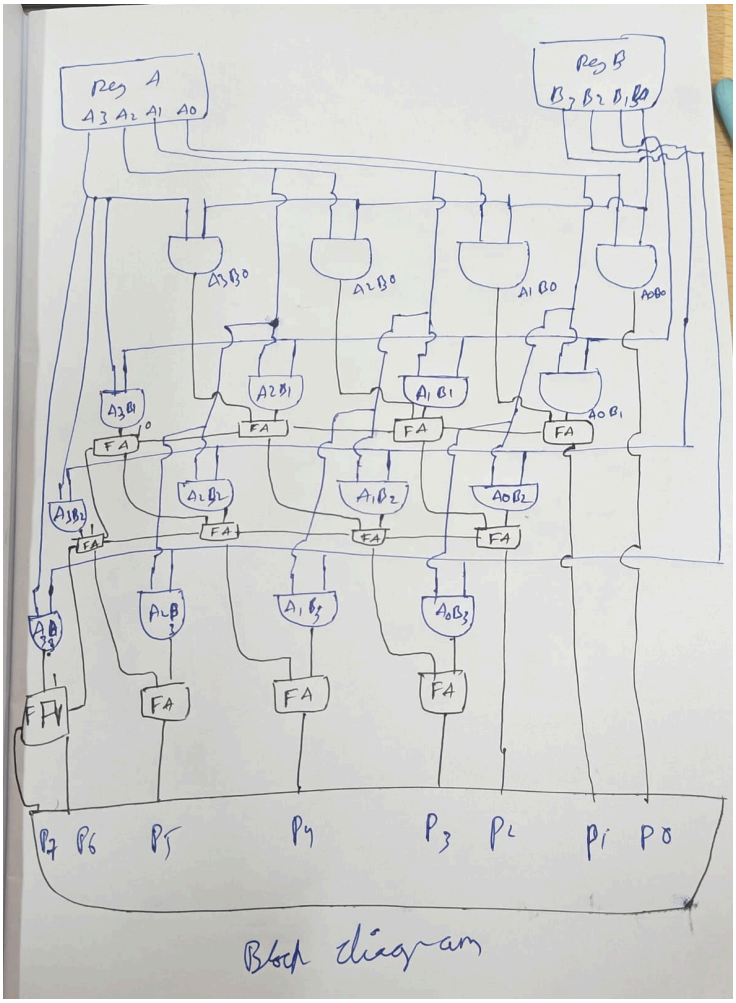
2.3 Structure of mult_array

Row	Operation	Output
Input registers	a_reg <= a, b_reg <= b on every rising clock edge	a_reg, b_reg
Row 0	4 AND gates (pp0 = a_reg AND b_reg[0])	p[0] = pp0[0], passes pp0[3:1] down
Row 1	4 AND gates + 4 full adders	p[1] = sum1[0], passes {c1[4], sum1[3:1]} down

Row	Operation	Output
Row 2	4 AND gates + 4 full adders	p[2] = sum2[0], passes {c2[4], sum2[3:1]} down
Row 3	4 AND gates + 4 full adders	p[3] = sum3[0], p[7:4] = {c3[4], sum3[3:1]}
Output register	p <= array result on every rising clock edge	p[7:0]

Component	Count
AND gates	16
Full adders	12
Flip-flops	16 (4 for a, 4 for b, 8 for p)

Block diagram:



Reg A and Reg B are 4-bit registers (4 flip-flops each), Reg P is an 8-bit register (8 flip-flops). The array has 16 AND gates and 12 full adders. clk and rst_n (asynchronous, active low) go to all 16 flip-flops.

2.4 Testbench and simulation

The testbench generates a 10 ns clock, holds rst_n low for 2 cycles, applies all 256 combinations of a (0 to 15) and b (0 to 15), waits 2 clock edges for each vector, compares p with a * b, and prints one summary line. Inputs are changed on the falling clock edge to avoid a race with the sampling edge.

Command	Purpose
iverilog -g2012 -o sim_rtl tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v	Compile
vvp sim_rtl	Run, prints the summary line
gtkwave mult.vcd	View waveform

Test	Result line
Correct design	PASS: 256 vectors checked, 0 errors
Error test (sum changed to a ^ b)	FAIL: 256 vectors checked, 95 errors

```
/foss/designs > iverilog -g2012 -o sim_rtl tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim_rtl
gtkwave mult.vcd
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:54: $finish called at 5188000 (1ps)

GTKWave Analyzer v4.0.0-prealpha (132a086) (w)1999-2022 BSI

[0] start time.
[5188000] end time.
WM Destroy
```

```
/foss/designs > cd /foss/designs

cp rtl/full_adder.v /tmp/full_adder.v.bak

sed -i 's/assign sum *=.*;/assign sum = a ^ b;/' rtl/full_adder.v

iverilog -g2012 -o sim_bad \
  tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v

vvp sim_bad | tail -3

cp /tmp/full_adder.v.bak rtl/full_adder.v

grep -n "assign sum" rtl/full_adder.v
MISMATCH: a=15 b=15 expected=225 got=181
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:54: $finish called at 5188000 (1ps)
cp: overwrite 'rtl/full_adder.v'? Y
9:    assign sum  = a ^ b ^ cin;
```

The error test temporarily changed the sum in full_adder.v to a ^ b. The first mismatch printed was a=15, b=15, expected=225, got=181. The original line (a ^ b ^ cin) was then restored and the testbench passed again.

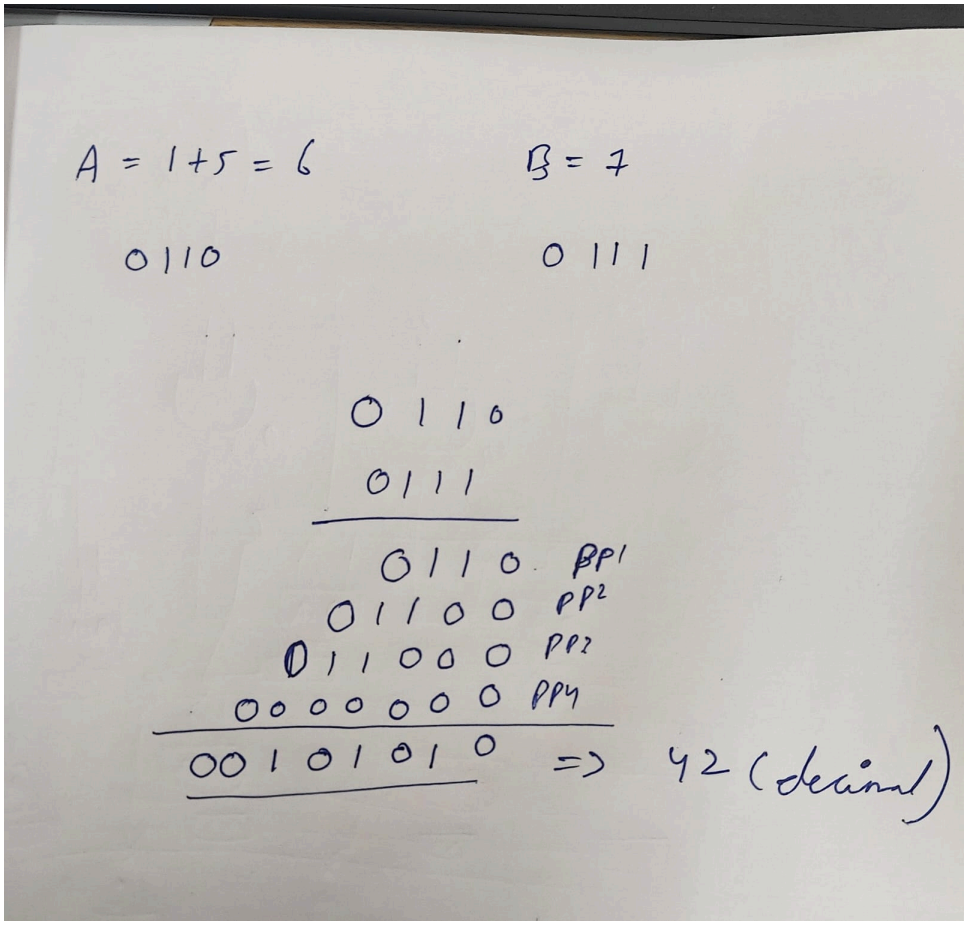
2.5 Hand-worked example

A = (last digit of a roll number) + 5. Roll number used: 2026702011 (K. Phalgun). Last digit = 1, so A = 1 + 5 = 6. B = 7. Product = 6 x 7 = 42.

Quantity	Binary	Decimal
A	0110	6
B	0111	7

Partial product	Rule	Value (shifted)
pp0	A AND b0 (b0 = 1)	0 1 1 0
pp1	A AND b1 (b1 = 1), shifted 1	0 1 1 0 0
pp2	A AND b2 (b2 = 1), shifted 2	0 1 1 0 0 0
pp3	A AND b3 (b3 = 0), shifted 3	0 0 0 0 0 0 0
Sum	pp0 + pp1 + pp2 + pp3	0 0 1 0 1 0 1 0

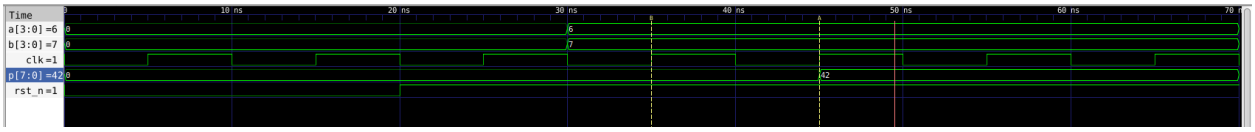
P = 00101010 (binary) = 42 (decimal) = 0x2A.



2.6 Waveform for the 6 x 7 example

Time	Event
0 to 20 ns	rst_n low, all registers cleared
30 ns	a = 6 and b = 7 applied (falling clock edge)
35 ns	Clock edge 1: a_reg and b_reg capture 6 and 7
45 ns	Clock edge 2: p captures 42

Latency = 2 clock cycles (edge 1 at 35 ns to edge 2 at 45 ns after the inputs are applied).



Waveform for 6 x 7 = 42. The marker labeled B is clock edge 1 (35 ns): a_reg and b_reg capture 6 and 7. The marker labeled A is clock edge 2 (45 ns): p = 42. Latency = 2 clock cycles.

3. Part B: Logic Synthesis with Yosys

3.1 Script and commands

synth/synth.tcl performs these steps (each written as yosys <command> in the Tcl script), with lib set to sky130_fd_sc_hd__tt_025C_1v80.lib.

Step	Command	Purpose
1	read_verilog rtl/full_adder.v rtl/mult_array.v	Read the design
2	synth -top mult_array -flatten	Generic synthesis, flatten full adders
3	dfflibmap -liberty $lib	Map flip-flops to SKY130 cells
4	abc -liberty $lib	Map logic to SKY130 gates
5	opt_clean -purge	Remove unused wires
6	tee -o synth/mult_array_stat.rpt stat -liberty $lib	Cell and area report
7	write_verilog -noattr synth/mult_array_netlist.v	Gate-level netlist

Run command	yosys -c synth/synth.tcl

3.2 Part B results table

Metric	Value
Total number of cells	68
Number of flip-flops	16
Total cell area (um^2)	785.754
Flip-flop area (um^2)	400.384 (16 x 25.024)
Flip-flop area (% of total)	50.96 %
The 3 most-used cell types and their counts	dfrtp_1 (16), nand2_1 (8), and2_0 (6, tied with xnor2_1 at 6)
Netlist simulation result	PASS: 256 vectors checked, 0 errors

3.3 Full cell list from mult_array_stat.rpt

Cell type	Count	Area (um^2)
sky130_fd_sc_hd__dfrtp_1	16	400.384
sky130_fd_sc_hd__nand2_1	8	30.029
sky130_fd_sc_hd__and2_0	6	37.536
sky130_fd_sc_hd__xnor2_1	6	52.550
sky130_fd_sc_hd__maj3_1	5	50.048
sky130_fd_sc_hd__nor2_1	4	15.014
sky130_fd_sc_hd__a21oi_1	3	15.014
sky130_fd_sc_hd__a22oi_1	3	22.522
sky130_fd_sc_hd__and4_1	2	17.517
sky130_fd_sc_hd__clkinv_1	2	7.507
sky130_fd_sc_hd__lpflow_isobufsrc_1	2	12.512
sky130_fd_sc_hd__mux2i_1	2	20.019
sky130_fd_sc_hd__o21ai_0	2	10.010
sky130_fd_sc_hd__xnor3_1	2	45.043
sky130_fd_sc_hd__a2111oi_0	1	8.758
sky130_fd_sc_hd__nand3_1	1	5.005
sky130_fd_sc_hd__nand4_1	1	6.256
sky130_fd_sc_hd__or3_1	1	6.256
sky130_fd_sc_hd__xor3_1	1	23.773
Total	**68**	**785.754**

```
/foss/designs > head -40 synth/mult_array_stat.rpt

7. Printing statistics.

=== mult_array ===

        +----------Local Count, excluding submodules.
        |        +-Local Area, excluding submodules.
        |        |
       59        - wires
       81        - wire bits
       15        - public wires
       37        - public wire bits
        5        - ports
       18        - port bits
       68  785.754 cells
        1    8.758   sky130_fd_sc_hd__a2111oi_0
        3   15.014   sky130_fd_sc_hd__a21oi_1
        3   22.522   sky130_fd_sc_hd__a22oi_1
        6   37.536   sky130_fd_sc_hd__and2_0
        2   17.517   sky130_fd_sc_hd__and4_1
        2    7.507   sky130_fd_sc_hd__clkinv_1
       16  400.384   sky130_fd_sc_hd__dfrtp_1
        2   12.512   sky130_fd_sc_hd__lpflow_isobufsrc_1
        5   50.048   sky130_fd_sc_hd__maj3_1
        2   20.019   sky130_fd_sc_hd__mux2i_1
        8   30.029   sky130_fd_sc_hd__nand2_1
        1    5.005   sky130_fd_sc_hd__nand3_1
        1    6.256   sky130_fd_sc_hd__nand4_1
        4   15.014   sky130_fd_sc_hd__nor2_1
        2    10.01   sky130_fd_sc_hd__o21ai_0
        1    6.256   sky130_fd_sc_hd__or3_1
        6    52.55   sky130_fd_sc_hd__xnor2_1
        2   45.043   sky130_fd_sc_hd__xnor3_1
        1   23.773   sky130_fd_sc_hd__xor3_1

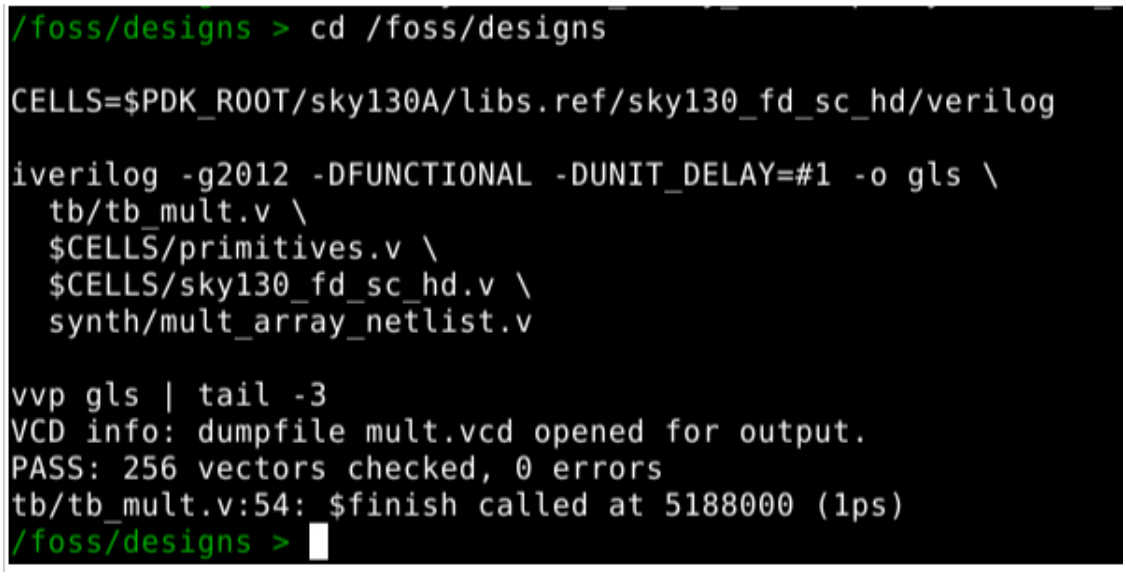
   Chip area for module '\mult_array': 785.753600
     of which used for sequential elements: 400.384000 (50.96%)
```

3.4 Netlist simulation (gate-level)

The synthesized netlist was simulated with the same testbench to prove that synthesis did not change the behaviour.

Run command
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls

Result
PASS: 256 vectors checked, 0 errors


/foss/designs > cd /foss/designs
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
tb/tb_mult.v \
$CELLS/primitives.v \
$CELLS/sky130_fd_sc_hd.v \
synth/mult_array_netlist.v
vvp gls | tail -3
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:54: $finish called at 5188000 (1ps)
/foss/designs >

4. Part C: Static Timing Analysis with OpenSTA

Typical corner only (tt_025C_1v80). Netlist: synth/mult_array_netlist.v.

4.1 Script and commands

Step	Command	Purpose
1	read_liberty $lib	Read the cell timing library
2	read_verilog synth/mult_array_netlist.v, link_design mult_array	Read and link the netlist
3	create_clock -name clk -period T [get_ports clk]	Define the clock
4	set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]	Input arrival time
5	set_output_delay 1.0 -clock clk [all_outputs]	Output required time
6	report_worst_slack -max	Worst setup slack
7	report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt	Critical path report

Run command	sta -no_splash sta/sta.tcl

4.2 Finding Tmin and Fmax

Quantity	Formula	Value
Worst slack at T = 10 ns (WNS)	from report_checks	7.027 ns
Tmin	T - WNS = 10 - 7.027	2.973 ns, rounded to 2.97 ns
Fmax	1000 / Tmin	336.7 MHz

4.3 Part C results table

Metric	Value
Worst slack at T = 10 ns (ns)	7.027
Tmin (ns)	2.97
Fmax (MHz)	336.7
Slack at T = Tmin (ns)	-0.003 (at T = 2.97 ns)
Critical path start point	_108_ (input register flip-flop, CLK to Q)

Metric	Value
Critical path end point	_103_ (output register p flip-flop, D pin)
Number of cells on the critical path	10(2 flip-flop + 8 combinational cells)
Latency (ns) = 2 x Tmin	5.94
Throughput (million results per second)	336.7

The slack at T = Tmin is -0.003 ns instead of exactly 0 because Tmin = 2.973 ns was rounded down to 2.97 ns (a rounding difference of 0.003 ns). This is close to zero as required.

4.4 Cells on the critical path, in order

No.	Instance	Cell	Delay (ns)	Arrival time (ns)
1	_108_/Q	sky130_fd_sc_hd__dfrtp_1	0.410	0.410
2	_050_/X	sky130_fd_sc_hd__and4_1	0.223	0.634
3	_061_/Y	sky130_fd_sc_hd__o21ai_0	0.195	0.828
4	_062_/Y	sky130_fd_sc_hd__nand3_1	0.140	0.969
5	_063_/X	sky130_fd_sc_hd__maj3_1	0.380	1.349
6	_069_/X	sky130_fd_sc_hd__xnor3_1	0.393	1.741
7	_074_/X	sky130_fd_sc_hd__maj3_1	0.369	2.110
8	_086_/X	sky130_fd_sc_hd__maj3_1	0.383	2.493
9	_087_/X	sky130_fd_sc_hd__maj3_1	0.349	2.842
10	_103_/D	sky130_fd_sc_hd__dfrtp_1	-	2.842 (data arrival)

Timing check at T = 10 ns	Value (ns)
Data arrival time	2.842
Library setup time	0.131
Data required time	9.869
Slack	7.027 (MET)

4.5 Full-adder cells on the critical path

Cell group	Cells	Count
Full-adder type (sum and carry)	xnor3_1 (1), maj3_1 (4)	5
Partial-product and merged logic	and4_1, o21ai_0, nand3_1	3
Flip-flop (clock to Q)	dfrtp_1 (start and end)	2

Comparison	Value
Estimate from Section 2 of the assignment (2N, N = 4)	about 8 full-adder delays
Full-adder cells counted on the path	5

The path crosses fewer full-adder cells than the estimate, explained in Q2.

5. Part D: Physical Design, RTL to GDSII

5.1 LibreLane configuration (pnr/config.json)

Key	Value
DESIGN_NAME	"mult_array"
VERILOG_FILES	["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"]
CLOCK_PORT	"clk"
CLOCK_PERIOD	7.5 (final passing value)
FP_SIZING	"absolute"
DIE_AREA	[0, 0, 150, 150]
Additional keys	DESIGN_IS_CORE: true, RUN_CTS: true, RUN_IRDROP_REPORT: false, SYNTH_STRATEGY: "AREA 0"

Run command	cd pnr, then librelane config.json

The starting period was 1.2 x Tmin = 1.2 x 2.97 = 3.56 ns, rounded up to the next 0.5 ns = 4.0 ns.

5.2 Run log

Run	CLOCK_PERIOD (ns)	Result
1	4.0	Failed: setup violations in the slow corners
2	4.5	Failed: setup violations
3	5.0	Failed: setup violations
4	5.5	Failed: setup violations
5	6.0	Failed: setup violations
6	6.5	Failed: setup violation
7	7.0	Failed: setup violation
8	7.5	Passed: DRC 0, LVS 0, antenna pass, setup slack +0.303 ns

5.3 Part D results table

Metric	Value
CLOCK_PERIOD used (ns)	7.5
Number of runs needed	8
Core area (um^2)	17,759.5
Cells after synthesis (from Part B)	68
Cells after place and route	1899 (total instances)
Kinds of cells added	Fill cells (1564), tap cells (245), timing repair buffers (20), clock buffers (3)
Wirelength (um)	2,423 (routed)
Setup slack (ns)	+0.303
Post-route Fmax (MHz)	139.0
Fmax from Part C (MHz)	336.7
DRC errors	0 (Magic: 0, KLayout: 0)
LVS errors	0

5.4 Post-route Fmax

Quantity	Formula	Value
Post-route Fmax	1000 / (CLOCK_PERIOD - setup slack) = 1000 / (7.5 - 0.303)	139.0 MHz
Part C Fmax	1000 / 2.97	336.7 MHz
Ratio	336.7 / 139.0	about 2.4 times higher in Part C

5.5 Other final metrics

Metric	Value
Hold slack (ns)	+0.130
Die area (um^2)	22,500 (150 um x 150 um)
Global-route wirelength estimate (um)	3,836
Routed wirelength (um)	2,423

Metric	Value
Antenna check	Passed

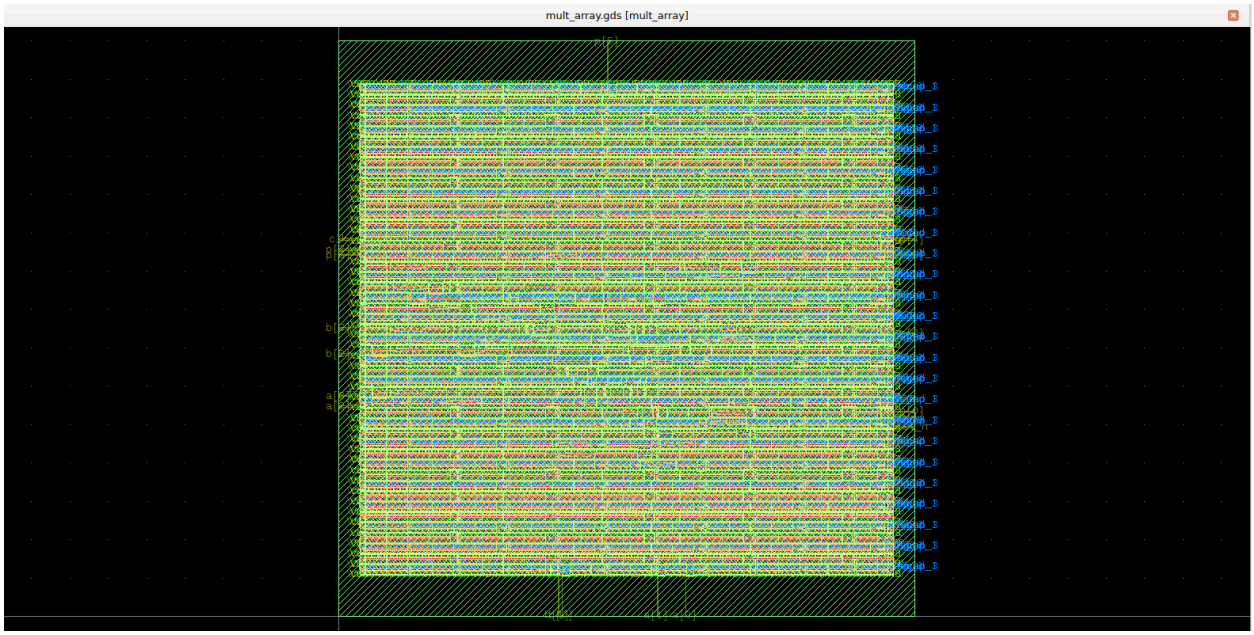
5.6 Cell count breakdown after place and route

Cell class	Count
Sequential cells (flip-flops)	16
Multi-input combinational cells	49
Inverters	2
Timing repair buffers	20
Clock buffers	3
Tap cells	245
Fill cells	1564
Total instances	**1899**

Comparison	Count
Cells after synthesis (Part B)	68
Logic and register cells after P&R (16 + 49 + 2 + 20 + 3)	90
Of which buffers added by the flow (20 + 3)	23
Physical-only cells added (tap + fill)	1809

The flow runs its own synthesis step, so the logic count differs from Part B by one cell (51 logic cells in the flow against 52 in Part B). The tap cells cover about 306.5 um^2.

5.7 Layout screenshots





6. Answers to the Report Questions

Q1. Logic cell count

Synthesis produced 68 cells in total, of which 16 are flip-flops, so the logic uses 68 - 16 = 52 cells. This is not 16 + 12 = 28 because a full adder is not a single library cell: its sum becomes an XOR/XNOR (xnor2, xnor3, xor3) and its carry becomes a majority gate (maj3) or an AND/OR network. ABC also merges AND gates and adders into complex cells such as a22oi, and4 and a2111oi, and adds inverters and buffers (clkinv, lpflow_isobufsrc). In addition, row 0 has no adders, and the first adder of each row has carry-in = 0, so its logic is simplified.

Item	Count
Total cells	68
Flip-flops	16
Logic cells (68 - 16)	52
Naive estimate (16 AND + 12 full adders)	28

Q2. Critical path

The critical path runs from an input register flip-flop (_108_) to an output register flip-flop (_103_). It has 10 cells: the starting flip-flop, 8 combinational cells and the capturing flip-flop. Only 5 of them are full-adder type cells (4 maj3 for carries and 1 xnor3 for a sum), which is fewer than the estimate of about 2N = 8 full-adder delays. The estimate assumes every full adder is a separate, unoptimized stage. In the synthesized netlist ABC merges the partial-product ANDs into complex gates, the carry-in = 0 adders reduce to simpler logic, and maj3 and xnor3 are compound cells that each take about 0.35 to 0.39 ns. The total path delay is 2.842 ns: 0.410 ns clock-to-Q plus about 2.43 ns of logic.

Item	Value
Start point	_108_ (input register)
End point	_103_ (output register p)
Cells on path	10
Full-adder type cells	5
Estimate (2N)	8
Total path delay	2.842 ns

Q3. Flip-flop area share

The flip-flops take 400.384 um^2 of the 785.754 um^2 total, which is 50.96 %. The percentage is large because the design is very small and has 16 registers (4 + 4 input bits and 8 output bits) around only 52

small logic cells. A dfrtp_1 flip-flop with asynchronous reset has an area of 25.024 um^2, about 7 times a nand2_1 (3.754 um^2), so the registers dominate the area even though they are few.

Item	Value
Flip-flop area	400.384 um^2
Total cell area	785.754 um^2
Flip-flop share	50.96 %
Area of one dfrtp_1	25.024 um^2
Area of one nand2_1	3.754 um^2

Q4. Why post-route Fmax is lower

The post-route Fmax is 139.0 MHz, against 336.7 MHz from Part C. Part C used the typical corner (1.8 V, 25 C) with an ideal clock and ideal wires, so it counted only gate delay. LibreLane signs off at the slow corners (for example ss_100C_1v60), where gates are slower, and it includes real wire resistance and capacitance from the routed layout. This is why the 4.0 to 7.0 ns attempts failed even though Part C predicted 2.97 ns. At the typical corner (nom_tt_025C_1v80) the post-route setup slack is +3.967 ns, which gives about 283 MHz (1000 / (7.5 - 3.967)), so most of the gap comes from the slow corner and the rest from the real wires and the clock tree. The flow added four kinds of cells:

Added cell type	Count	Why it is needed
Timing repair buffers	20	Fix slew, fanout and hold problems on nets; they add some delay
Clock buffers	3	Clock tree so the clock reaches all 16 flip-flops at nearly the same time
Tap cells	245	Tie the wells to power and ground to prevent latch-up; they have no signal connections
Fill cells	1564	Fill empty space in the rows to keep continuous wells and meet density rules; they have no signal connections

Tap and fill cells do not affect timing; the slowdown comes from the slow corner, the wire delay, and the added buffers. The 245 tap cells most likely include the end-cap cells (decap_3) placed at the row ends.

Fmax comparison	Value
Part C (typical corner, ideal wires and clock)	336.7 MHz
Post-route (slow corners, real wires, clock tree)	139.0 MHz

Q5. DRC and LVS

A DRC (design rule check) verifies that the layout obeys the foundry's geometric manufacturing rules, such as minimum width, spacing, enclosure and density of each layer. An LVS (layout versus schematic) check extracts the circuit from the layout and verifies that it matches the synthesized netlist, with the same devices and connections. Both must pass: a layout can be DRC-clean but wired differently from the intended circuit (wrong function), or match the netlist but violate the rules so that it cannot be fabricated reliably. In this design both pass.

Check	What it verifies	Result
DRC	Layout follows manufacturing geometry rules	0 errors (Magic and KLayout)
LVS	Layout circuit equals the netlist	0 errors
Antenna	Metal antenna ratio rules	Passed

Q6. Scaling to an 8-bit multiplier

Using the formulas from Section 2 of the assignment: an N-bit array multiplier uses N^2 AND gates and N(N - 1) full adders, and its longest path is roughly 2N full-adder delays.

Quantity	N = 4	N = 8	Change
AND gates (N^2)	16	64	4 times
Full adders N(N - 1)	4 x 3 = 12	8 x 7 = 56	about 4.7 times
Longest path (2N full-adder delays)	8	16	2 times

The number of full adders grows roughly with the square of the width, while the delay grows roughly linearly. Estimated Tmin for 8 bits (using the Part C numbers): the logic part of the 4-bit critical path is 2.842 - 0.410 = 2.432 ns; doubling it gives about 4.86 ns; adding the clock-to-Q (0.410 ns) and setup (0.131 ns) gives about 5.4 ns, so Fmax would drop from 336.7 MHz to about 185 MHz. This is an estimate based on the 2N rule.

Estimate for 8 bits	Value
Logic delay (2 x 2.432 ns)	about 4.86 ns
Tmin (4.86 + 0.410 + 0.131)	about 5.4 ns
Fmax	about 185 MHz